



CHANDIGARH UNIVERSITY
APEX INSTITUTE OF TECHNOLOGY



ASSIGNMENT 2

Subject: Computer Networks

Subject Code: 24CST-339

Submitted By:

Name: Kamal Sharma

UID: 24BAI70380

Class: 24BAIT_KRG_1B

Submitted To:

Mr. Alok Singh Rathore

Q1. What is an IP Address? Why do we need classes in case of IPv4 address?
Explain each class, mention how many hosts and networks are possible in each class. Also explain subnetting and supernetting with examples.

Answer:

An **IP Address (Internet Protocol Address)** is a unique logical address assigned to a device on a network. It is used to identify the device and enable communication between different devices.

An **IPv4 address** is a **32-bit address**, usually written in four decimal parts called octets.
Example: 192.168.1.10

Why do we need classes in IPv4?

IPv4 addresses were originally divided into different **classes** to efficiently allocate IP addresses according to the size of the network. Each class provided a different number of networks and hosts.

	Class	First Octet	Default Mask	Networks	Hosts per Network
A		1–126	/8	126	16,777,214
B		128–191	/16	16,384	65,534
C		192–223	/24	2,097,152	254
D		224–239	—	Multicast	—
E		240–255	—	Experimental	—

Note: In Classes A, B and C, some addresses are reserved, so the usable numbers are slightly lower than the total possible addresses.

- **Class A:** Used for very large networks. The first 8 bits represent the network and remaining 24 bits represent hosts.
- **Class B:** Used for medium-sized networks. The first 16 bits represent the network and remaining 16 bits represent hosts.
- **Class C:** Used for small networks. The first 24 bits represent the network and remaining 8 bits represent hosts.
- **Class D:** Used for multicast communication.
- **Class E:** Reserved for experimental/research purposes.

Subnetting

Subnetting is the process of dividing one large network into smaller networks called **subnets**.

Example:

Suppose we have 192.168.1.0/24, which provides 254 usable host addresses.

If we divide it into /26 subnets, we get **4 smaller subnets**, each having **62 usable hosts**.

This helps in better utilization of IP addresses and reduces network traffic.

Supernetting

Supernetting is the process of combining multiple smaller networks into one larger network.

Example:

Four Class C networks:

192.168.0.0/24

192.168.1.0/24

192.168.2.0/24

192.168.3.0/24

can be combined into:

192.168.0.0/22

Supernetting is useful for **route aggregation** and reducing the number of routing table entries.

Q2. What is the difference between Link State and Distance Vector routing? Explain how these two algorithms help in identifying the least cost path with examples.

Answer:

Routing is the process of finding the best path for sending data from a source to a destination.

Link State Routing

In **Link State Routing**, each router maintains information about the entire network topology. Routers share information about their directly connected links with other routers.

The router then uses **Dijkstra's algorithm** to calculate the shortest/least-cost path.

Example:

Suppose:

- $A \rightarrow B = 2$
- $B \rightarrow C = 3$
- $A \rightarrow C = 8$

Possible paths from A to C:

- Direct: $A \rightarrow C = 8$
- Through B: $A \rightarrow B \rightarrow C = 2 + 3 = 5$

Therefore, the least-cost path is:

$A \rightarrow B \rightarrow C = 5$

Distance Vector Routing

In **Distance Vector Routing**, each router maintains a table containing the distance (cost) to different destinations and the next hop. Routers exchange this information with their neighboring routers.

It commonly uses the **Bellman-Ford algorithm** to calculate the least-cost path.

Example:

Suppose router A knows:

- $A \rightarrow B = 2$
- $B \rightarrow C = 3$

A can calculate the cost to C through B as:

$$2 + 3 = 5$$

If the direct path $A \rightarrow C$ costs 8, A chooses the path through B because its cost is lower.

Difference between Link State and Distance Vector

Link State	Distance Vector
Knows the complete network topology	Knows information mainly from neighbors
Uses Dijkstra's algorithm	Uses Bellman-Ford algorithm
Exchanges link-state information	Exchanges routing tables/distance information
Generally converges faster	Generally converges slower
Requires more memory and processing	Requires less memory and processing

Both methods help routers determine the **least-cost path** between a source and destination.

Q3. What is congestion in a network? Why do we need to control congestion? Explain the difference between Leaky Bucket and Token Bucket with examples. Also, explain why do we need to do fragmentation?

Answer:

Congestion

Congestion occurs when too much data is sent into a network and the network cannot handle the amount of traffic efficiently.

As a result, routers may experience:

- Packet loss
- Increased delay
- Reduced network performance
- Buffer overflow

Why do we need congestion control?

Congestion control is needed to prevent excessive traffic from overwhelming the network. It helps to:

- Reduce packet loss
- Reduce delays
- Improve network performance
- Maintain efficient utilization of network resources

Leaky Bucket

The **Leaky Bucket** algorithm controls the rate at which packets leave a network.

Packets enter a bucket/queue and leave at a **fixed rate**. If the bucket becomes full, additional packets are discarded.

Example:

Suppose the bucket can hold 5 packets and sends **1 packet per second**.

If 8 packets arrive suddenly:

- 5 can be stored.

- They are transmitted at 1 packet/second.
- The remaining packets may be discarded.

Thus, Leaky Bucket produces a **smooth and constant output rate**.

Token Bucket

The **Token Bucket** algorithm allows packets to be sent whenever tokens are available. Tokens are generated at a fixed rate and stored in the bucket.

If tokens have accumulated, the network can temporarily handle a **burst of traffic**.

Example:

Suppose tokens are generated at 2 tokens/second and 5 tokens have accumulated. A device can send a burst of 5 packets immediately, provided it has enough tokens.

Difference between Leaky Bucket and Token Bucket

Leaky Bucket	Token Bucket
Sends data at a fixed rate	Allows variable/burst transmission
Smooths traffic	Controls traffic while allowing bursts
Packets may be discarded when bucket is full	Tokens accumulate when there is no traffic
Does not naturally support bursts	Supports bursty traffic

Fragmentation

Fragmentation is the process of dividing a large IP packet into smaller packets.

It is needed when a packet is larger than the **Maximum Transmission Unit (MTU)** supported by a network.

Example:

Suppose a packet is **4000 bytes**, but the network can carry a maximum of **1500 bytes** per packet.

The original packet must be divided into smaller fragments so that they can travel through the network.

The fragments are later **reassembled at the destination**.

Therefore, fragmentation allows large packets to travel across networks with different maximum packet sizes.

